

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

*I G. Naveen Kumar Reddy_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

SIMATS ENGINEERING

Assessment Tool 1 – Application Based Questions Computer Architecture (CSA12)

Name	KAVALI BALAJI
Register No.	192525202
Course Outcome	CO1 (BL4) – CPU, Memory, Bus Structures and I/O Components

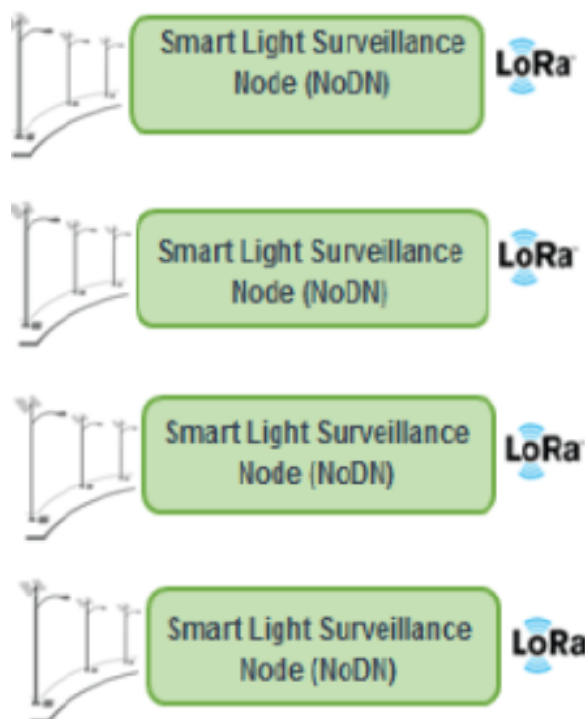
Q1. Smart City Surveillance System

Theory

A smart city surveillance system continuously captures live video from multiple cameras and transfers the data through I/O devices to main memory. The CPU fetches, decodes, and executes instructions to analyze video frames and detect threats. During peak hours, heavy traffic between CPU, memory, and I/O creates bottlenecks. A single-bus architecture allows only one transfer at a time, whereas a multi-bus architecture supports parallel communication and improves throughput. Performance can be enhanced using cache memory, DMA, instruction pipelining, multicore processors, and high-speed buses.

A **Smart City Surveillance System** is an advanced security solution that uses technologies such as **Artificial Intelligence (AI)**, **the Internet of Things (IoT)**, **cloud computing**, and **CCTV cameras** to monitor public areas in real time. It helps city authorities improve public safety, manage traffic, detect crimes, and respond quickly to emergencies. Unlike traditional surveillance systems that only record video, smart surveillance systems can automatically analyze live footage, detect suspicious activities, recognize faces and vehicle number plates, and generate instant alerts.

Schematic Diagram



Example Code

```
while(1){  
    frame = CaptureVideo();  
    StoreInMemory(frame);  
    cpu_fetch(frame);  
    cpu_decode();  
    cpu_execute();  
    if(threat_detected) DisplayAlert();  
}
```

Advantages

- Improves performance
- Reduces execution delay
- Efficient memory utilization
- Increases reliability

FUTURE SCOPE

- **Advanced AI** and Machine Learning Integration
- 5G and IoT-Based Smart Surveillance
- Drone-Based Monitoring and Surveillance
- Predictive Crime Analysis and Prevention
- Integration with Smart Traffic and Emergency Response Systems

Conclusion

The **Smart City Surveillance System** improves public safety by using modern technologies such as **CCTV cameras, IoT sensors, Artificial Intelligence (AI), and cloud computing** for continuous real-time monitoring. It helps detect suspicious activities, manage traffic, respond quickly to emergencies, and reduce crime. AI-based analytics minimize human effort and increase the accuracy of threat detection. The system also stores surveillance data securely for future investigation. Overall, a Smart City Surveillance System makes cities **safer, smarter, and more efficient**, while supporting better decision-making for law enforcement and city administrators.

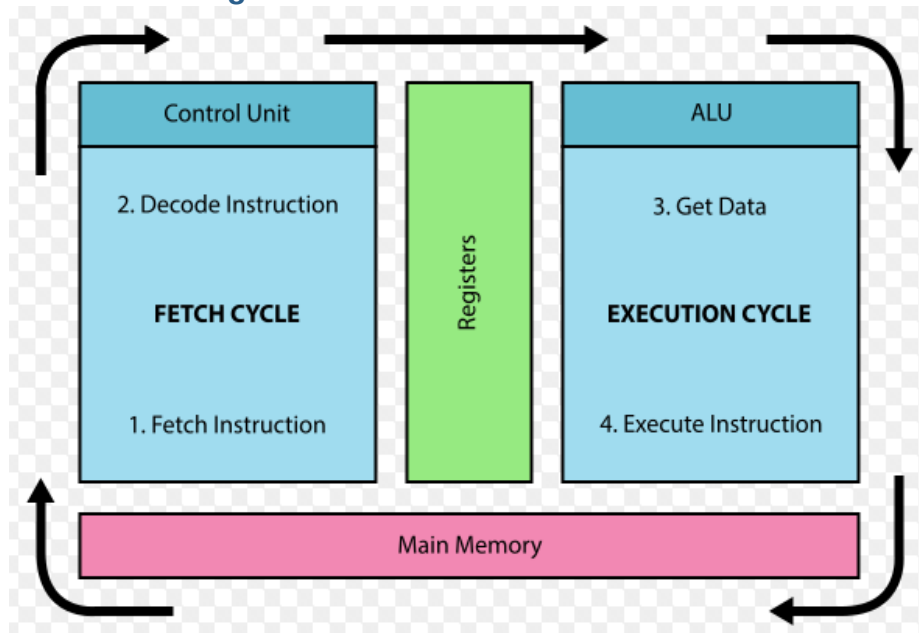
Q2. Real-Time Stock Trading Application

Theory

A real-time stock trading application executes millions of instructions every second. Frequent memory accesses, cache misses, and branch instructions increase CPI, resulting in higher execution time. Each instruction passes through Fetch, Decode, Execute, and Write Back stages. CPU performance is improved using cache memory, branch prediction, instruction pipelining, superscalar execution, and compiler optimization.

A Real-Time Stock Trading Application is a software platform that allows users to buy and sell stocks instantly using live market data. It provides real-time stock prices, portfolio management, market analysis, and secure online transactions, enabling investors to make informed investment decisions.

Schematic Diagram



Example Code

```
for(i=0; i<n; i++){  
  fetch();  
  decode();  
  execute();  
  writeback();  
}
```

Advantages

- Improves performance
- Reduces execution delay
- Efficient memory utilization
- Increases reliability

Future Scope

- AI-Based Stock Price Prediction
- Blockchain for Secure Transactions
- Integration with Global Stock Markets

- Personalized Investment Recommendations
- Advanced Real-Time Risk Analysis

Conclusion

The **Real-Time Stock Trading Application** provides a fast, secure, and efficient platform for buying and selling stocks using live market data. It enables investors to make informed decisions through real-time price updates, portfolio tracking, and market analysis. By integrating advanced technologies such as AI, cloud computing, and secure payment systems, the application improves trading accuracy, transparency, and user experience. Overall, it simplifies stock trading, enhances investment management, and supports the growing demand for digital financial services.

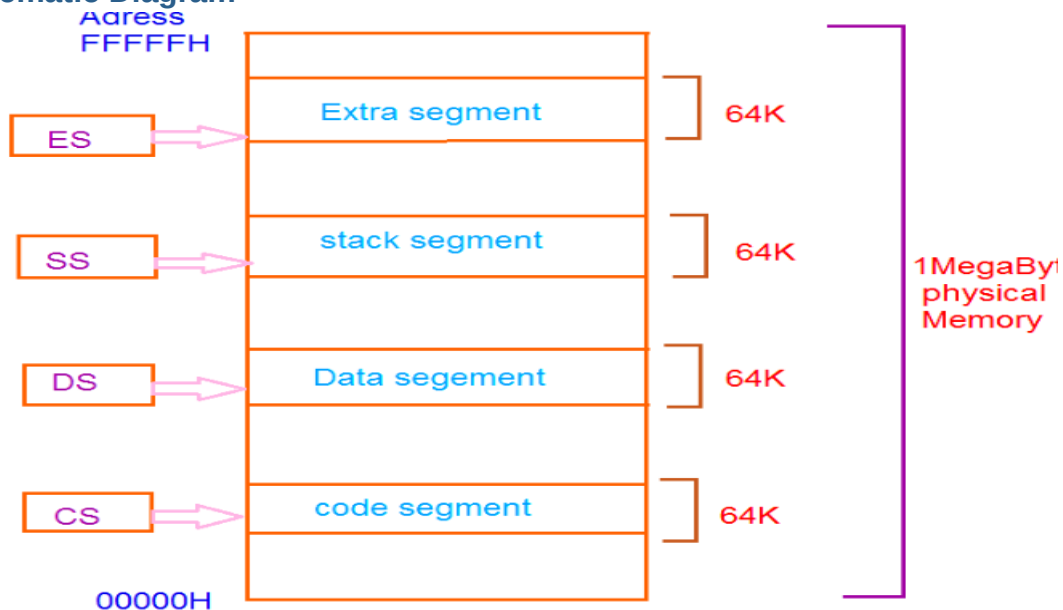
Q3. 8086 Segmented Memory

Theory

The Intel 8086 microprocessor uses segmented memory architecture to access 1 MB of memory. Memory is divided into Code, Data, Stack, and Extra segments. Physical Address = Segment $\times$ 10H + Offset. Correct segment management and addressing modes ensure reliable execution.

The **8086 microprocessor** uses a **segmented memory architecture** to access up to **1 MB (2^{20} bytes)** of memory, even though it has only **16-bit registers**. Memory is divided into segments of **64 KB** each, making memory management more organized and efficient. A physical address is generated by combining a **segment address** and an **offset address**. The four main memory segments are **Code Segment (CS)** for program instructions, **Data Segment (DS)** for data storage, **Stack Segment (SS)** for stack operations, and **Extra Segment (ES)** for additional data storage.

Schematic Diagram



Example Code

```
MOV AX, DATA
MOV DS, AX
MOV AL, [1000H]
ADD AL, 05H
MOV [1001H], AL
MOV AH, 4CH
INT 21H
```

Advantages

- Improves performance
- Reduces execution delay
- Efficient memory utilization
- Increases reliability

Future scope

- Supports **1 MB memory** using segmented addressing.
- Each segment has a maximum size of **64 KB**.
- Physical Address = **Segment × 10H + Offset**.
- Four segments: **CS, DS, SS, and ES**.
- Improves memory organization and program execution.

Conclusion

The 8086 segmented memory architecture enables the processor to access up to 1 MB of memory by combining segment and offset addresses. It organizes memory into separate segments for code, data, stack, and extra data, making memory management more efficient. This approach improves program organization, supports larger applications, and enhances the overall performance and flexibility of the 8086 microprocessor.

These architectural techniques improve system performance, speed, and reliability.